

A Stochastic Dynamic Pricing Engine for General Aviation: Integrating Logistic Regression, Constrained Optimization, and Markov Decision Processes

Flight Pricing Engine — Technical Report

Kamran Sohrab

Tuesday — April 28, 2026

Abstract

This report describes the design, mathematical foundations, and computational Implementation of a three-layer dynamic pricing engine developed for a general aviation aircraft operating under Federal Aviation Regulation Part 61.113 cost-sharing constraints. The system integrates three methodologies drawn from my learnings in the principles of industrial engineering: (1) an online logistic regression model trained via stochastic gradient descent with L2 regularization that estimates passenger acceptance probability as a function of quoted price and contextual features; (2) a constrained nonlinear program that maximizes expected revenue per seat subject to hard aircraft weight, fuel capacity, and cost-recovery constraints, with the fuel-sourcing sub-problem formulated as a closed-form linear program; and (3) a tabular Q-learning agent that models the sequential pricing problem as a Markov Decision Process and learns an optimal policy over a discretized state-action space. All three layers update online after every pricing session, yielding a system that becomes progressively more accurate as flight history accumulates. Aircraft parameters in this report are fixed to a specific default four-seat aircraft with a maximum usable weight of 1,241.23 lbs, 72-gallon fuel capacity, and an operating cost of \$115.00 per tach hour. However, aircraft and model parameters may be changed to suit the needs of potential users.

Contents

1	Introduction	3
2	Problem Context and Aircraft Parameters	3
2.1	Operational Constraints	3
2.2	Operating Cost Model	4
3	Mathematical Model	5
3.1	Layer 1 — Logistic Regression Demand Model	5
3.1.1	Model Specification	5
3.1.2	Online Learning via Stochastic Gradient Descent	5
3.2	Layer 2 — Constrained Revenue Optimization	6
3.2.1	Fuel Sourcing Linear Program	6
3.2.2	Weight and Balance Feasibility Constraint	7
3.2.3	Revenue Maximization Nonlinear Program	7
3.3	Layer 3 — Markov Decision Process and Q-Learning	8
3.3.1	MDP Formulation	8
3.3.2	State Space Discretization	8
3.3.3	Action Space	9
3.3.4	Q-Learning Update Rule	9
3.3.5	Exploration Strategy	9
3.4	Cross-Layer Price Blending	10
4	Computational Architecture and Data Flow	10
4.1	Module Structure	10
4.2	Persistent State	11
4.3	Session Execution Flow	11
5	Online Learning Dynamics and System Convergence	12
5.1	Cold-Start Behaviour	12
5.2	Convergence of the Logistic Regression	13
5.3	Convergence of Q-Learning	13
5.4	The Feedback Loop	13
6	Limitations and Future Work	14
6.1	Current Limitations	14
6.2	Directions for Future Work	14
6.3	Current Works in Progress	15
7	Conclusion	16

1 Introduction

Pricing decisions in transportation markets have long been studied as problems of revenue management under demand uncertainty. Airlines, ride-sharing platforms, and freight carriers employ sophisticated dynamic pricing algorithms that adjust fares in real time based on observed demand, load factors, and booking lead times. General aviation, however, operates under a fundamentally different regulatory and economic structure: private pilots may share pro-rata operating costs with passengers under FAR 61.113, but may not charge fares that exceed their proportional share of those costs in a manner that constitutes compensation for carriage. Commercial/Charter Pilots (without an Air Operator Certificate) may receive compensation for flying services, given the only service they provide is the flight. Within this constraint, a pilot wishing to recover costs in a transparent, data-driven way faces a genuinely interesting optimization problem.

This report presents a pricing engine that treats each flight quote as a decision under uncertainty, where the key unknown is whether a prospective passenger will accept a given price. The system is grounded in three methodological pillars: statistical demand learning via logistic regression, constrained revenue optimization via nonlinear programming, and long-run policy improvement via reinforcement learning. These three pillars correspond directly to the core content of four of my industrial engineering courses: *Analysis of Data*, *Numerical Methods*, *Deterministic Models of Optimization*, and *Stochastic Processes and Applications*.

The remainder of this report is organized as follows. Section 2 describes the physical and regulatory problem context and defines the aircraft parameters. Section 3 develops the mathematical model for each of the three layers. Section 4 details the computational architecture and data flow. Section 5 presents the system’s feedback loop and online learning dynamics. Section 6 discusses limitations and ideas/directions for future work. Section 7 concludes.

2 Problem Context and Aircraft Parameters

2.1 Operational Constraints

The default aircraft under consideration in this case is a four-seat single-engine piston aircraft. The pilot occupies one seat, leaving at most three revenue-generating passenger seats. The following hard constraints govern every flight decision and are enforced before any pricing computation proceeds. (see Table 1)

Table 1: Fixed aircraft parameters used throughout the pricing engine.

Parameter	Symbol	Value	Unit
Maximum usable weight	$W_{\max}$	1241.23	lbs
Maximum fuel capacity	$G_{\max}$	72.0	gallons
Fuel weight	w_f	6.0	lbs/gal
Cruise fuel burn	B	12.0	gal/hr
VFR fuel reserve	r	0.75	hours
Maximum payload	W_{payload}	200.0	lbs
Pilot weight (assumed)	W_{pilot}	160.0	lbs
Cost per tach hour	C_{tach}	115.00	\$/tach hr
Tach-to-flight-hour ratio	τ	0.80	dimensionless
Available passenger seats	N_{seats}	3	seats

2.2 Operating Cost Model

The tachometer hour is the fundamental billing unit for piston aircraft. Because the tachometer runs proportionally to engine RPM rather than elapsed time, it accumulates more slowly than a wall clock or a hobbs clock during cruise. The long-run empirical conversion of the default aircraft has been identified as 0.80 tach hours per one flight hour. For a flight of duration T hours (one way), the number of tach hours incurred is:

$$T_{\text{tach}} = \tau \cdot T = 0.80 \cdot T \quad (1)$$

For a round trip, this quantity (equation 1) is roughly doubled for simplicity. We can denote this relationship with the function $\Omega(z)$, defined as a piecewise function given a boolean input, z ; *true* if the flight is round trip, and *false* otherwise:

$$\Omega(z) = \begin{cases} 2 & \text{if } z = \text{true} \\ 1 & \text{if } z = \text{false} \end{cases} \quad (2)$$

The aircraft operating cost for a given flight is then:

$$C_{\text{aircraft}} = [T_{\text{tach}} \cdot \Omega(z)] \cdot C_{\text{tach}} = [0.80 \cdot T \cdot \Omega(z)] \cdot 115.00 \quad (3)$$

The minimum fuel required to complete the flight safely, including the mandatory VFR reserve, is:

$$G_{\text{req}} = B \cdot (T + r) = 12.0 \cdot (T + 0.75) \quad (4)$$

where $B = 12.0$ gal/hr is the cruise burn rate and $r = 0.75$ hr is the reserve. The total fuel cost depends on the sourcing strategy (developed in Section 3.2) and is denoted C_{fuel} . The total operating cost is:

$$C_{\text{ops}} = C_{\text{aircraft}} + C_{\text{fuel}} \quad (5)$$

3 Mathematical Model

3.1 Layer 1 — Logistic Regression Demand Model

3.1.1 Model Specification

The fundamental quantity of interest is the probability that a passenger accepts a quoted price p given the context of the flight. This is modelled as a binary outcome — accept ($y = 1$) or deny ($y = 0$) — conditioned on a feature vector $\mathbf{x}$. The logistic regression model specifies:

$$P(\text{accept} \mid \mathbf{x}) = \sigma(\boldsymbol{\beta}^\top \mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\beta}^\top \mathbf{x}}} \quad (6)$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the logistic sigmoid function. The feature vector $\mathbf{x} \in \mathbb{R}^6$ and coefficient vector $\boldsymbol{\beta} \in \mathbb{R}^6$ are defined as follows (see Table 2):

Table 2: Feature vector definition and initial coefficient values.

Index j	x_j	Description	Initial β_j
0	1	Intercept / bias term	2.000
1	p	Quoted price per passenger (\$)	−0.010
2	T	One-way flight duration (hours)	−0.100
3	d	Booking lead time (days before departure)	0.005
4	n	Number of paying passengers	−0.050
5	rt	Round-trip indicator (1 = round trip, 0 = one-way)	0.100

Thus, the entire logistic regression model is:

$$P(\text{accept} \mid \mathbf{x}) = \sigma(\boldsymbol{\beta}^\top \mathbf{x}) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 p + \beta_2 T + \beta_3 d + \beta_4 n + \beta_5 rt)}} \quad (7)$$

The initial coefficients reflect prior beliefs about passenger behaviour: higher prices reduce acceptance probability ($\beta_1 < 0$), longer flights are less attractive at a given price ($\beta_2 < 0$), passengers booking further in advance are marginally more accepting ($\beta_3 > 0$), and groups are slightly more price-sensitive than individuals ($\beta_4 < 0$). The intercept $\beta_0 = 2.0$ encodes a general prior leaning toward acceptance, ensuring the model does not reject all prices before data is observed.

3.1.2 Online Learning via Stochastic Gradient Descent

The model is trained online; coefficients are updated immediately after each new accept/deny observation rather than in batches. The update rule is derived from the cross-entropy loss

with an L2 regularization penalty:

$$\mathcal{L}(\boldsymbol{\beta}) = -[y \log \hat{p} + (1 - y) \log(1 - \hat{p})] + \frac{\lambda}{2} \|\boldsymbol{\beta}\|^2 \quad (8)$$

where $\hat{p} = \sigma(\boldsymbol{\beta}^\top \mathbf{x})$ is the current model prediction and λ is the regularization coefficient. The stochastic gradient descent update for a single observation $(\mathbf{x}, y)$ is:

$$\beta_j \leftarrow \beta_j + \alpha [(y - \hat{p}) x_j - \lambda \beta_j] \quad \text{for } j = 1, \dots, 5 \quad (9)$$

$$\beta_0 \leftarrow \beta_0 + \alpha(y - \hat{p}) \quad (\text{intercept, no regularization}) \quad (10)$$

The term $(y - \hat{p})$ is the prediction error. When the model predicts correctly, the error is small and the update is minimal. When the model is wrong, the error is large and the coefficients shift substantially. The L2 term $\lambda \beta_j$ shrinks coefficients toward zero, preventing any single feature from dominating; a safeguard particularly important when the dataset is small.

Table 3: Logistic regression hyperparameters.

Hyperparameter	Symbol	Value	Role
Learning rate	α	0.05	SGD step size
L2 regularization	λ	0.01	Penalizes large coefficients
Minimum samples	$N_{\min}$	5	Records before model activates

3.2 Layer 2 — Constrained Revenue Optimization

3.2.1 Fuel Sourcing Linear Program

Before the ticket price can be optimized, the minimum-cost fuel loading strategy must be determined. Let c_o and c_d denote the price per gallon at origin and destination respectively, and let g_o and g_d denote the gallons loaded at each airport. The fuel sourcing problem for a round trip is:

$$\min_{g_o, g_d} \quad c_o g_o + c_d g_d \quad (11)$$

$$\text{subject to} \quad g_o \geq G_{\text{req}, \text{out}} \quad (12)$$

$$g_d \geq G_{\text{req}, \text{ret}} \quad (13)$$

$$g_o \leq G_{\text{wb}} \quad (14)$$

$$g_d \leq G_{\text{max}} = 72.0 \quad (15)$$

$$g_o, g_d \geq 0 \quad (16)$$

where G_{wb} is the maximum gallons loadable at departure without violating the weight constraint (derived in Section 3.2.2), and $G_{\text{req}, \text{out}} = G_{\text{req}, \text{ret}} = B(T + r)$. Because the objective is separable, the LP decouples and the optimal solution is obtained by evaluating two closed-form strategies:

- **Strategy A (origin full):** Load $g_o = \min(G_{\text{req,out}} + G_{\text{req,ret}}, G_{\text{wb}})$ at origin, nothing at destination. Cost = $g_o \cdot c_o$.
- **Strategy B (split fueling):** Load $g_o = G_{\text{req,out}}$ at origin, $g_d = G_{\text{req,ret}}$ at destination. Cost = $g_o c_o + g_d c_d$.

The strategy with the lower total cost is selected. For one-way flights, $g_d = 0$ trivially and the problem reduces to $g_o = \min(G_{\text{req,out}}, G_{\text{wb}})$. This assumes the flight will terminate at the destination airport, and the pilot will not return to a potential home/base airport; or will quote a following flight departing the airport the previous flight terminated at.

3.2.2 Weight and Balance Feasibility Constraint

Every proposed flight configuration must satisfy the hard weight constraint before any pricing computation proceeds. Given n paying passengers with average weight w_{pax} , payload W_{payload} , pilot weight W_{pilot} , and fuel gallons g_o loaded at departure, the total ramp weight is:

$$W_{\text{total}} = w_f g_o + n w_{\text{pax}} + W_{\text{payload}} + W_{\text{pilot}} \quad (17)$$

The feasibility condition is the inequality constraint:

$$W_{\text{total}} \leq W_{\text{max}} = 1,241.23 \text{ lbs} \quad (18)$$

If this constraint is violated, the pricing session terminates. The maximum gallons that can be loaded within the weight limit is obtained by solving for g_o :

$$G_{\text{wb}} = \frac{W_{\text{max}} - n w_{\text{pax}} - W_{\text{payload}} - W_{\text{pilot}}}{w_f} \quad (19)$$

This quantity is passed as the upper bound on g_o in constraint (14).

3.2.3 Revenue Maximization Nonlinear Program

Given the operating cost C_{ops} and the demand model $P(\text{accept} \mid \mathbf{x})$, the expected revenue per passenger at price p is:

$$R(p) = p \cdot P(\text{accept} \mid \mathbf{x}) \quad (20)$$

We seek the price that maximizes expected revenue subject to the cost-recovery floor and a market ceiling:

$$p^* = \arg \max_p p \cdot P(\text{accept} \mid \mathbf{x}) \quad (21)$$

$$\text{subject to } p \geq p_{\text{floor}} = \frac{C_{\text{ops}}}{n} \cdot (1 + \delta_{\text{min}}) \quad (22)$$

$$p \leq p_{\text{ceil}} = \frac{C_{\text{ops}}}{n} \cdot \kappa \quad (23)$$

where $\delta_{\text{min}} = 0.05$ is the minimum margin (5% above per-seat cost) and $\kappa = 5.0$ is the market ceiling multiplier. The revenue function $R(p) = p \sigma(\boldsymbol{\beta}^\top \mathbf{x}(p))$ is empirically unimodal over $[p_{\text{floor}}, p_{\text{ceil}}]$ when $\beta_1 < 0$, which holds by construction. The optimization proceeds in two phases:

1. **Grid search:** Evaluate $R(p)$ at $N_{\text{grid}} = 200$ equally spaced prices across $[p_{\text{floor}}, p_{\text{ceil}}]$ and identify the approximate global maximum.
2. **Scalar refinement:** Apply Brent’s method (`scipy.optimize.minimize_scalar`) in a narrow interval around the grid maximum for sub-cent precision.

When $n_{\text{updates}} < N_{\text{min}}$, the model defaults $P(\text{accept}) = 0.5$ and returns p_{floor} as the conservative recommendation.

3.3 Layer 3 — Markov Decision Process and Q-Learning

3.3.1 MDP Formulation

The pricing problem is cast as a finite Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where:

- $\mathcal{S}$ is the finite state space (defined in Section 3.3.2).
- $\mathcal{A}$ is the finite action space: a set of discrete price multipliers.
- $\mathcal{P}(s' | s, a)$ is the transition probability, implicitly¹ learned through experience.
- $\mathcal{R}(s, a)$ is the expected reward: revenue minus cost if accepted; zero if denied.
- $\gamma = 0.92$ is the discount factor.

The objective is to find the optimal policy $\pi^*(s)$ that maximizes expected discounted cumulative reward:

$$V^*(s) = \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t \mathcal{R}(s_t, a_t) \mid s_0 = s, \pi \right] \quad (24)$$

This is characterized by the Bellman optimality equation:

$$V^*(s) = \max_a \left[\mathcal{R}(s, a) + \gamma \sum_{s'} \mathcal{P}(s' | s, a) V^*(s') \right] \quad (25)$$

¹ Because the underlying system is continuous and partially observed, these probabilities are not known explicitly and are instead approximated through experience via Q-learning. We assume that the discretized state representation preserves an approximate Markov property, such that future states depend primarily on the current discretized state and action.

3.3.2 State Space Discretization

The continuous problem variables are discretized into a finite state space $\mathcal{S} = \mathcal{S}_1 \times \mathcal{S}_2 \times \mathcal{S}_3 \times \mathcal{S}_4$ of cardinality $|\mathcal{S}| = 4 \times 4 \times 3 \times 4 = 192$.

Table 4: State space dimensions and discretization breakpoints.

Dimension	Variable	Bins	Breakpoints
$\mathcal{S}_1$: Acceptance rate	Recent route acceptance rate	4	$[0\%, 25)$, $[25, 50)$, $[50, 75)$, $[75, 100\%]$
$\mathcal{S}_2$: Lead time	Days from quote to departure	4	Same-day, $[1, 3)$, $[3, 14)$, 14+ days
$\mathcal{S}_3$: Load factor	Passengers / available seats	3	$\leq 33\%$, $(33\%, 67\%]$, $> 67\%$
$\mathcal{S}_4$: Season	Calendar quarter	4	Q1, Q2, Q3, Q4

3.3.3 Action Space

The action space consists of $N_A = 12$ discrete actions, each representing a price multiplier relative to the NLP-optimal price p^* computed in Layer 2. Action k is mapped to a multiplier:

$$m_k = 0.60 + \frac{(1.40 - 0.60)k}{N_A - 1}, \quad k = 0, 1, \dots, 11 \quad (26)$$

yielding multipliers from 0.60 (aggressive discount) to 1.40 (40% premium). The suggested price is $p_Q = m_k \cdot p^*$.

3.3.4 Q-Learning Update Rule

Because the transition probabilities $\mathcal{P}(s' | s, a)$ are unknown a priori, we apply Q-learning, a model-free, off-policy reinforcement learning algorithm that estimates the action-value function:

$$Q^*(s, a) = \mathcal{R}(s, a) + \gamma \sum_{s'} \mathcal{P}(s' | s, a) \max_{a'} Q^*(s', a') \quad (27)$$

After each pricing session, given that state s was observed, action a was taken, reward r was realized, and the system transitioned to state s' , the Q-table is updated by the Temporal Difference (TD) rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha_Q \underbrace{\left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]}_{\text{TD error}} \quad (28)$$

A positive TD error means the outcome exceeded expectations; the Q-value increases. A negative TD error means it fell short; the Q-value decreases.

3.3.5 Exploration Strategy

The agent uses an ε -greedy strategy: with probability ε , a random action is chosen (exploration); otherwise the greedy action $\arg \max_a Q(s, a)$ is chosen (exploitation). After each episode, ε decays:

$$\varepsilon_{t+1} = \max(\varepsilon_{\min}, \varepsilon_t \cdot \delta_\varepsilon) = \max(0.02, \varepsilon_t \cdot 0.97) \quad (29)$$

After approximately 109 episodes, ε reaches its floor of 0.02, meaning the agent explores only 2% of the time in steady state.

Table 5: MDP and Q-learning hyperparameters.

Hyperparameter	Symbol	Value	Role
Discount factor	γ	0.92	Future reward weighting
Q-learning rate	α_Q	0.15	Bellman update step size
Initial epsilon	ε_0	0.25	Initial exploration rate
Minimum epsilon	$\varepsilon_{\min}$	0.02	Exploration floor
Epsilon decay	δ_ε	0.97	Multiplicative decay per episode
Price action bins	N_A	12	Discrete price multipliers

3.4 Cross-Layer Price Blending

The NLP optimizer (Layer 2) and the Q-learning agent (Layer 3) each produce an independent price recommendation. These are blended into a single final price using a weight that grows logarithmically with the Q-agent’s experience:

$$w_Q = \min(0.50, 0.05 \cdot \ln(1 + n_{\text{episodes}})) \quad (30)$$

$$p_{\text{final}} = (1 - w_Q) p_{\text{NLP}}^* + w_Q p_Q \quad (31)$$

When the Q-agent has zero episodes, $w_Q = 0$ and $p_{\text{final}} = p_{\text{NLP}}^*$ entirely. As experience accumulates the MDP’s contribution grows, reaching its cap of 50% at $n_{\text{episodes}} = e^{10} - 1 \approx 22,026$. The blended price is clamped to $[p_{\text{floor}}, p_{\text{ceil}}]$ to ensure the cost-recovery floor is never violated.

4 Computational Architecture and Data Flow

4.1 Module Structure

The system is organized into nine Python modules with strict separation of concerns. Each module corresponds to a single conceptual responsibility (see Table 6).

Table 6: Module structure and key components.

Module	Responsibility	Key Components
<code>config.py</code>	All constants and hyperparameters	Tables 1, 3, 5 values
<code>demand_model.py</code>	Layer 1: logistic regression	<code>LogisticDemandModel</code> , <code>update()</code>
<code>optimizer.py</code>	Layer 2: fuel LP + NLP pricing	<code>solve_fuel_lp()</code> , <code>optimize_price()</code>
<code>mdp_agent.py</code>	Layer 3: Q-learning MDP	<code>QLearningAgent</code> , <code>update()</code>
<code>pricer.py</code>	Session orchestration	<code>run_pricing_session()</code>
<code>history.py</code>	JSON persistence + route statistics	<code>FlightHistory</code> , <code>append()</code>
<code>reports.py</code>	Analytics and diagnostics	<code>run_report()</code>
<code>utils.py</code>	CLI I/O and formatting	<code>prompt_float()</code> , etc.
<code>main.py</code>	Single entry point with menu	<code>main()</code>

4.2 Persistent State

Two JSON files constitute the system’s persistent memory:

- **`flight_history.json`:** An append-only log of every completed pricing session. Each record contains all input parameters, intermediate computations, the final price quoted, and the accept/deny outcome. This file grows indefinitely and is the source of truth for route acceptance rates and analytical reporting.
- **`model_state.json`:** A single overwritten snapshot of the current model state, containing the logistic regression β vector and update count (Section 3.1), and the Q-table, ϵ , and episode count (Section 3.3). This file is rewritten in full after every session. Its size is bounded and constant regardless of the number of sessions processed.

4.3 Session Execution Flow

A complete pricing session executes the following steps in sequence.

1. **Input collection.** The user provides origin, destination, departure date/time, round-trip flag, number of passengers, average passenger weight, payload weight, fuel prices at origin and destination, and estimated flight duration.
2. **Context derivation.** The route key is constructed (e.g., KCMi-KORD). Lead time d is computed as calendar days between the current date and the requested departure. The recent acceptance rate for the route is retrieved from `flight_history.json`.

3. **Weight budget computation.** G_{wb} is computed via Equation (19) to determine the maximum gallons loadable within the 1,241.23 lb limit.
4. **Fuel LP solution.** The fuel sourcing LP (Equations 11–16) is solved by evaluating Strategies A and B and selecting the minimum-cost option.
5. **Weight feasibility check.** W_{total} is computed via Equation (17) and compared against W_{max} . If infeasible, the session terminates.
6. **Operating cost.** C_{ops} is computed via Equation (5) using the LP-optimal fuel cost.
7. **MDP state construction and Q-suggestion.** The four-dimensional state s is constructed via the discretization functions in Table 4. The Q-agent returns its greedy price multiplier m_k and the implied suggestion $p_Q = m_k \cdot p_{\text{floor}}$.
8. **NLP revenue optimization.** The optimizer sweeps the price grid and refines to obtain p_{NLP}^* (Equation 21). If insufficient data exists, the floor price is returned.
9. **Price blending.** w_Q is computed via Equation (30) and p_{final} via Equation (31), clamped to $[p_{\text{floor}}, p_{\text{ceil}}]$.
10. **Results display.** The weight breakdown, fuel recommendation, operating cost, model status, NLP price, blended final price, $P(\text{accept})$, expected revenue, and expected profit are presented.
11. **Outcome recording.** The user records whether the quote was accepted or denied. The realized reward is:

$$r = \begin{cases} p_{\text{final}} \cdot n - C_{\text{ops}} & \text{if accepted} \\ 0 & \text{if denied} \end{cases} \quad (32)$$

12. **Layer 1 update.** One SGD step is performed via Equations (9) and (10), updating all six β coefficients.
13. **Layer 3 update.** The next state s' is constructed. One Bellman update is applied to $Q(s, a)$ via Equation (28). ε decays via Equation (29).
14. **Persistence.** The full session record is appended to `flight_history.json`. `model_state.json` is overwritten with the updated β and Q-table.

5 Online Learning Dynamics and System Convergence

5.1 Cold-Start Behaviour

When the system has processed fewer than $N_{\text{min}} = 5$ records, the logistic regression is in a cold-start state. In this regime, $P(\text{accept} \mid \mathbf{x}) = 0.5$ for all p , rendering the revenue function $R(p) = 0.5 \cdot p$ linearly increasing in p . The optimizer therefore returns p_{floor} as the recommended price (the minimum price that ensures cost recovery) which is the appropriate conservative choice when demand elasticity is unknown. The Q-agent is also uninformative

during this period (all Q-values initialized to zero), so $p_{\text{final}} = p_{\text{floor}}$. This cold-start behaviour ensures the system never underprices a flight due to model uncertainty; it may leave revenue on the table, but it never operates at a loss.

5.2 Convergence of the Logistic Regression

Under standard conditions for online SGD, namely that the learning rate α satisfies the Robbins–Monro conditions and the data are drawn i.i.d. from a fixed distribution, the β coefficients converge almost surely to the maximum likelihood estimate of the true logistic regression parameters. In this implementation, however, a constant learning rate $\alpha = 0.05$ is used. As a result, the parameter vector β does not converge exactly, but instead fluctuates within a neighborhood of the optimum. This behavior is appropriate in a non-stationary environment where passenger behavior may evolve over time. In practice the data are not i.i.d. (the price quoted depends on the current model, creating feedback), but the L2 regularization term stabilizes the trajectory by bounding $\|\beta\|$. The regularization coefficient $\lambda = 0.01$ was chosen to prevent overfitting while allowing meaningful coefficient movement in a data-sparse environment.

5.3 Convergence of Q-Learning

Tabular Q-learning converges to $Q^*(s, a)$ under the conditions that every state–action pair is visited infinitely often and the learning rate satisfies $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$. The ε -greedy strategy with $\varepsilon \geq \varepsilon_{\min} = 0.02$ ensures that all state–action pairs continue to be visited in the limit. With a fixed learning rate $\alpha_Q = 0.15$, strict convergence is not guaranteed, but fixed rates are standard in non-stationary environments, which this is, since the logistic model itself is evolving. The discount factor $\gamma = 0.92$ reflects the understanding that a pricing decision today has consequences for future demand: consistently overpricing a route trains passengers to expect refusal, while underpricing may attract demand but sacrifices margin.

5.4 The Feedback Loop

The three layers form a closed feedback loop over time. The logistic regression calibrates $P(\text{accept})$ more accurately as data accumulates, which reshapes the NLP revenue curve and shifts p_{NLP}^* . The Q-agent observes the rewards that flow from the blended price and updates its policy accordingly. The blending weight w_Q grows with Q-agent experience, gradually transferring pricing authority from the purely myopic NLP optimizer to the long-run policy of the MDP. Each layer informs the others, and the system as a whole improves with every flight.

6 Limitations and Future Work

6.1 Current Limitations

- **Sample size.** The logistic regression and Q-table both require substantial data to become well-calibrated. A pilot flying 50 trips per year may not see convergence for several years on low-frequency routes.
- **Feature scope.** The current feature vector does not include weather, day of week, competitor pricing, or special events — all of which may materially affect passenger willingness to pay.
- **Passenger heterogeneity.** The model treats all passengers on a given flight as having the same price sensitivity. In reality, individual passengers may have very different valuations.
- **Passenger memory.** The model treats each passengers as a new passenger during each quote. In reality, passengers may return as repeated customers, and their loyalty and willingness to pay premiums likely differ to new passengers.
- **Regulatory scope.** The system does not enforce the FAR 61.113 pro-rata cost-sharing ceiling. The user is responsible for ensuring compliance.
- **Static aircraft parameters.** The model assumes fixed fuel burn and operating costs. In practice these vary with altitude, loading, and maintenance cycles.

6.2 Directions for Future Work

- **Bayesian logistic regression.** Replacing the point-estimate SGD model with a Bayesian formulation would provide uncertainty quantification. The system could report a credible interval on $P(\text{accept})$, enabling risk-aware pricing.
- **Deep Q-Network (DQN).** As the feature space expands, the tabular Q-table becomes intractable. A neural network function approximator could generalize across states and handle continuous state representations.
- **Multi-route transfer learning.** Logistic regression weights trained on high-volume routes could warm-start models on new routes, reducing the cold-start period.
- **Simulation-based validation.** A discrete-event simulation of passenger arrival and acceptance behaviour would allow offline evaluation of the pricing policy before deployment, enabling hyperparameter tuning without real-world cost.

6.3 Current Works in Progress

- **Passenger Directory.** Passengers may return as repeated customers, and their loyalty and willingness to pay differ from new passengers. In this directory, passenger data — such as name, a unique identifier, weight, and a feature vector $\mathbf{z}$ — are stored. The feature vector $\mathbf{z}$ will be used to store and update prior specific β values regarding passenger booking biases, and would be passed into the Logistic Regression Demand Model to generate a dynamic price. The Passenger Directory will be an overwrite and append JSON file, stored in the `data` folder, along with `flight_history.json` and `model_state.json`. *This feature requires an entire rework of all 3 three model layers. While rollout is not expected to be soon, the feature is expected to rollout in the next version release of this model.*
- **Electronic Flight Bag Connection/API.** Most pilots use Electronic Flight Bag applications (most commonly, *Foreflight* and *Garmin Pilot*) to assist in preflight responsibilities, and calculating all applicable weight and balance, and fuel calculations. Connecting to a user’s specific account instance would allow for a streamlined model, while utilizing real world flight data rather than relying on idealized assumptions on quotes. *This feature is second in the work in progress workflow and is currently being developed. This feature likely will not rollout with the next version release of this model.*
- **Loyalty Program, Discounts, Flight Credits.** By allowing users to create and manage Loyalty Programs, Discounts, and Flight Credit, users can draw extra demand to their services. Using this loyalty, the model can confidently raise prices with the increased expectation that loyal passengers would rather earn benefits and use their discounts/flight credits than spend their money elsewhere. Loyalty status, Discounts, and outstanding flight credits will be stored in the `passenger_directory.json` file, and will be reflected in the passenger feature vector $\mathbf{z}$. *This feature has not entered development.*

7 Conclusion

This report has presented a three-layer dynamic pricing engine for general aviation that integrates logistic regression, constrained optimization, and reinforcement learning into a unified, self-improving system. Each layer addresses a distinct aspect of the pricing problem: Layer 1 estimates how likely a passenger is to accept a given price based on observed history; Layer 2 finds the price that maximizes expected revenue subject to hard aircraft safety constraints; and Layer 3 learns a long-run policy that accounts for the sequential, stochastic nature of the pricing environment.

The system is grounded in industrial engineering methodology at every level. The logistic regression and SGD update rules draw directly from statistical learning theory as taught in *Analysis of Data* and *Numerical Methods*. The fuel LP and revenue NLP are canonical applications of the constraint-handling and objective-function frameworks from *Deterministic Models of Optimization*. The MDP formulation, Bellman equation, and Q-learning algorithm are core content of *Stochastic Processes and Applications*. The result is not a collection of disconnected techniques but an integrated system where each component is motivated by the structure of the problem and informs the behaviour of the others.

The system is intentionally designed for transparency and interpretability: every coefficient, Q-value, and intermediate calculation is accessible through the diagnostics report, allowing the user to audit the model’s reasoning at any point. As data accumulates, the system becomes progressively more accurate, shifting from conservative cost-plus pricing toward a nuanced, demand-aware policy that balances short-run revenue maximization with long-run route development.